



Tests Logiciel



Objectifs

- ▶ Identifier les différents types de tests
- ▶ Développer des Tests Unitaires
- ▶ Développer des Tests d'Intégration

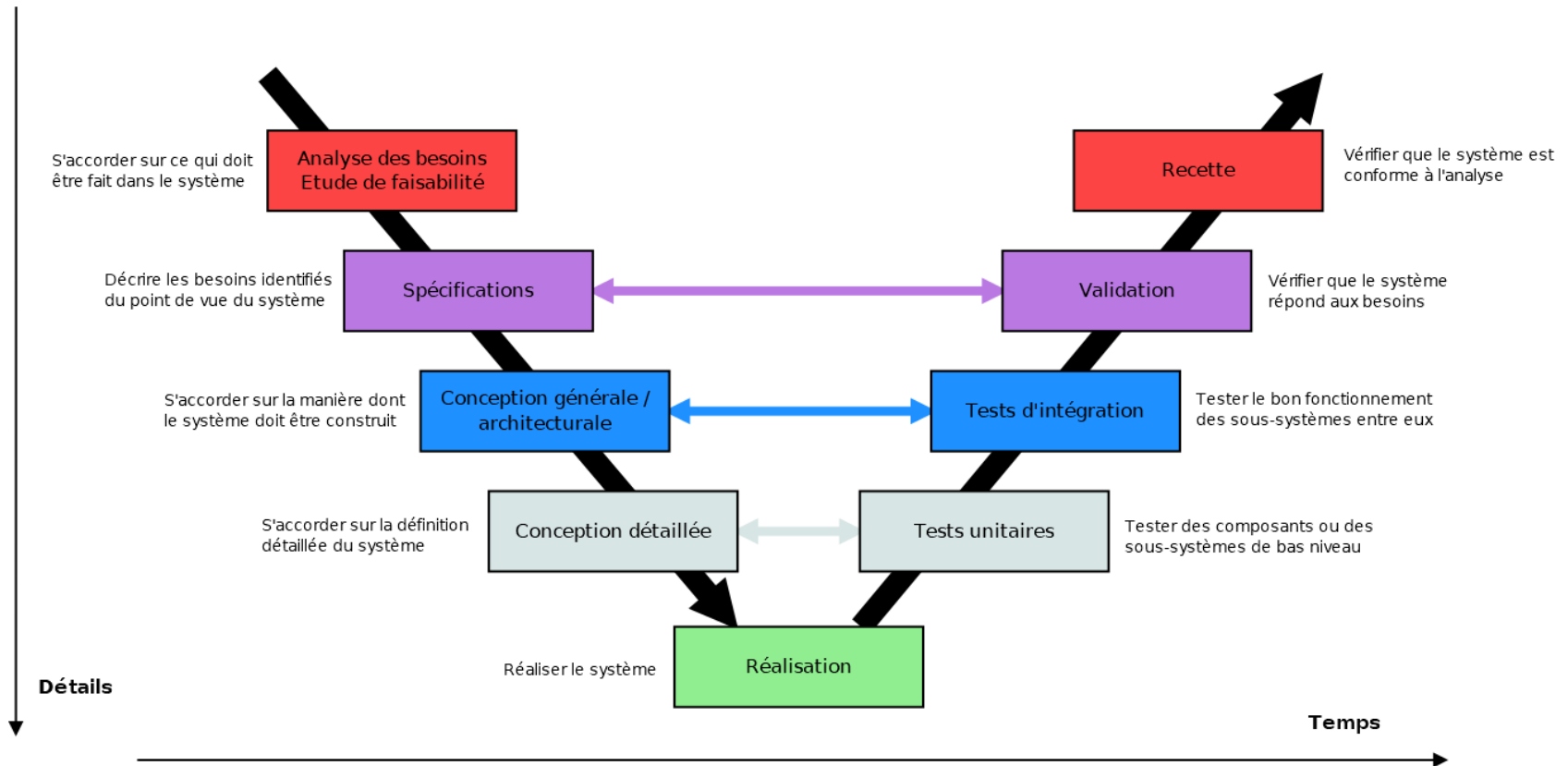


Définitions

- ▶ **Test logiciel** = processus qui consiste à évaluer et à vérifier qu'un système ou un composant fait ce qu'il est censé faire
- ▶ **Vérification** = vérifie que l'on a bien fait le logiciel
Est-ce que le logiciel fonctionne correctement ?
- ▶ **Validation** : vérifie que l'on a fait le bon logiciel
Est-ce que le logiciel réalise les fonctionnalités attendues ?

Dans le cycle en V

- ▶ A chaque étape de « conception » correspond une étape de test

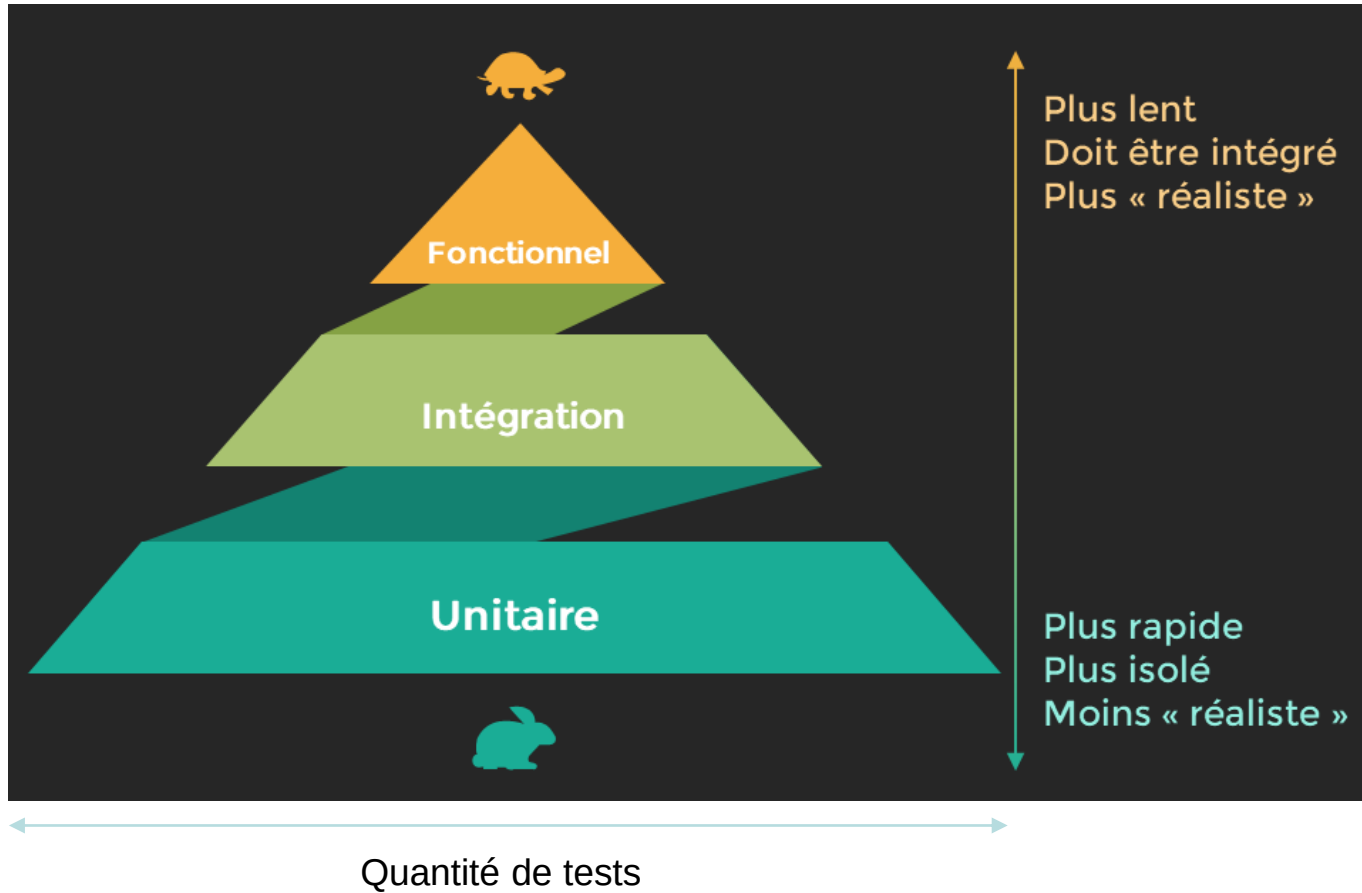


Dans un cycle agile

- Dans un cycle de développement incrémental, les tests sont joués sur chaque incrément



La pyramide des tests

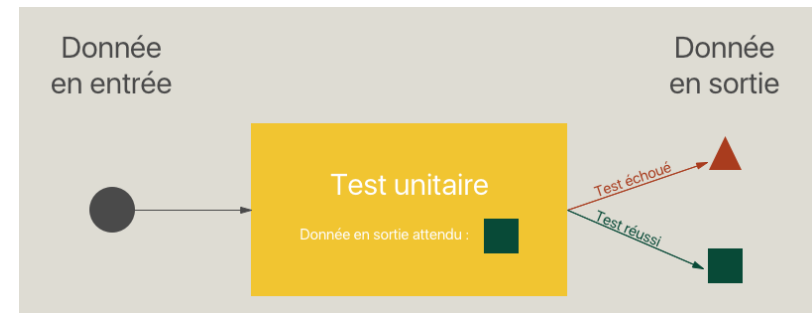


Tests manuels // automatisés

- ▶ **Manuels** : le testeur entre les données de test au moyen d'une interface simple, exécute l'action, observe les résultats obtenus et les compare avec ceux attendus
 - ⇒ *fastidieux, erreur humaine possible, difficiles à rejouer*
- ▶ **Automatisés** : on utilise un outil pour coder des tests qui pourront être facilement exécutés et rejoués
 - ⇒ *rejouables rapidement, à chaque modification du code pour mettre en évidence d'éventuelles régressions, élément clé d'un processus d'Intégration Continue*

Tests Unitaires

- ▶ Vérifier le fonctionnement d'une unité (méthode)
- ▶ Détecter les erreurs au plus tôt
- ▶ Eviter les régressions
- ▶ Tests boîte blanche, petits, rapides, ciblés, réalisés pendant le développement, par les développeurs de l'application
- ▶ Isoler la classe testée $\Rightarrow$ utiliser des simulacres
- ▶ Valider la conception détaillée



Tests d'Intégration

- ▶ Tester **une chaine de composants**
- ▶ Détecter les erreurs non décelables par les TU
- ▶ Démontrer que des unités interagissent de manière correcte, stable et cohérente et délivrent les résultats attendus
- ▶ Valider **la conception générale**



Tests fonctionnels

- ▶ Vérifier **une fonctionnalité complète** au regard **des spécifications**
- ▶ Basés sur **les scénarios d'utilisation**
- ▶ Tests **boite noire**
- ▶ Plus difficile à automatiser
- ▶ Vérifier aussi les exigences non fonctionnelles : interface, performance, utilisabilité, sécurité, robustesse

Tests d'acceptation ou recette

- ▶ Valider que le produit dans son ensemble répond aux besoins et attentes des utilisateurs
- ▶ Réalisés par les utilisateurs finaux
- ▶ Exécutés sur l'infrastructure du client

Caractéristiques des TU

Fast Rapide	Un test unitaire doit s'exécuter rapidement afin d'avoir un retour quasi immédiat Rapides à écrire $\pm$ mauvaise conception (refactoring)
Isolé Indépendant	Tester une classe à la fois Quand un test échoue, on doit savoir pourquoi Utiliser des simulacres (mocks) pour isoler la classe de ses dépendances
Répérable	Construire des tests qui n'interfèrent pas les uns avec les autres, assez autonomes pour donner toujours le même résultat
Self-validation	Utiliser des assertions pour auto-vérifier L'exécution ne doit laisser aucun doute sur le succès ou l'échec du test
Timely Thorough (approfondi)	Ecrire les tests le plus tôt possible Tester largement, explorer tous les chemins de code, les limites

Organiser les Tests Unitaires

- ▶ Créer un projet de test
 - ⇒ Basé sur un **framework de tests** (**xUnit**)
 - ⇒ Référence le projet contenant les classes à tester
 - ⇒ Utilise **des simulacres** (**moq**) afin d'isoler les tests
- ▶ Une classe de test par classe à tester
 - ⇒ Regroupant **une série de tests**
 - ⇒ Un **test** (**Fact**) cible **une seule méthode**
 - ⇒ Plusieurs tests sont nécessaires pour couvrir **tous les chemins de code**



Les Tests Unitaires documentent le code

Structure d'un Test Unitaire

```
[Fact]
✓ | 0 références
public async void GetBookByIdShouldBeNotFound()
{
    //Arrange
    int id = It.IsAny<int>();
    ILibraryService libraryService = Mock.Of<ILibraryService>();
    Mock.Get(libraryService)
        .Setup(lib => lib.GetBookByIdAsync(id))
        .ReturnsAsync(null as Book);

    BooksController controller = new BooksController(libraryService);

    //Act
    IActionResult actualResult = await controller.GetBookById(id);

    //Assert
    Assert.NotNull(actualResult as NotFoundResult);
}
```



Convention de nommage des tests

Visual Studio - Explorateur de tests

The screenshot shows the 'Explorateur de tests' window in Visual Studio. The top toolbar indicates 22 successful tests, 22 passed, and 0 failed. The main pane displays a tree view of test results. The 'Récapitulatif du groupe' pane on the right shows the summary for the 'TestsUnitaires' group.

Test	Durée
TestsUnitaires (22)	778 ms
TestsUnitaires (22)	778 ms
AuthorsControllerTest (5)	161 ms
BooksControllerTest (12)	314 ms
CreateBookEmptyShouldBadRequestWhenTitleEmpty	85 ms
CreateBookShouldBeBadRequestWhenAuthorNotFound	40 ms
CreateBookShouldBeOK	29 ms
DeleteBookShouldBeNoContent	39 ms
DeleteBookShouldBeNotFound	8 ms
GetBookByIdShouldBeNotFound	7 ms
GetBookByIdShouldBeOK	5 ms
GetBooksShouldBeOK	8 ms
UpdateBookShouldBeBadRequestWhenAuthorNotFound	< 1 ms
UpdateBookShouldBeBadRequestWhenIdInvalid	85 ms
UpdateBookShouldBeNotFound	4 ms
UpdateBookShouldBeOK	4 ms
StockServiceTest (5)	303 ms

Récapitulatif du groupe

TestsUnitaires

Tests dans le groupe: 22

⌚ Durée totale: 778 ms

Résultats

✓ 22 Réussite

► Couverture du code

- ⇒ Mesure le taux de code source exécuté d'un programme quand une suite de test est lancée
- ⇒ Ne vous concentrez pas sur une couverture de code à 100%
- ⇒ Ce n'est qu'un indicateur technique, qui doit être examiné dans le contexte de l'application, et qui ne prouve en rien la qualité du code et des tests unitaires

► Live Unit Testing

- ⇒ Exécute des tests automatiquement en **temps réel**

► Fixture :

- ⇒ Morceau de code qui permet de fixer un environnement logiciel pour exécuter des tests
- ⇒ Ainsi, l'environnement est toujours le même à chaque exécution des tests
- ⇒ Permet de répéter les tests et d'avoir toujours les mêmes résultats.
- ⇒ Exemple : initialiser une base de données

A savoir

- ▶ [les-10-commandements-des-tests-unitaires/](#)
- ▶ [mock - Quickstart](#)

